

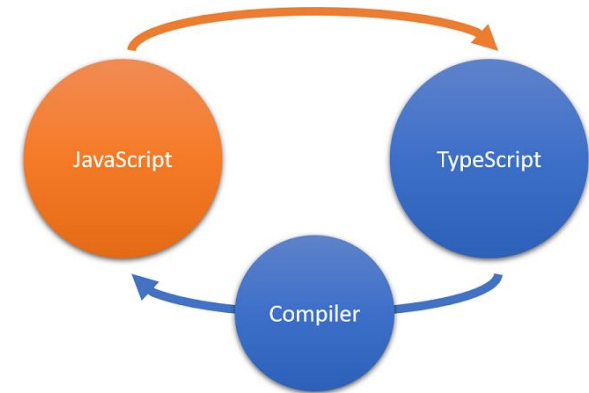
JavaScript - Fortgeschritten

Web-Engineering II

Prof. Dr. Sebastian von Klinski

TypeScript

- Programmiersprache 2012 von Microsoft entwickelt
- Basiert auf ECMAScript-6-Standard
- Erweiterung (Obermenge) von JavaScript
 - JavaScript-Code ist auch gültiger TypeScript-Code
 - Alle JavaScript-Programme können auch unter TypeScript verwendet werden
- Wird nach JavaScript kompiliert
- TypeScript-Kompiler ist ein Transkompiler (Source-Code nach Source-Code)
- Kapseln von Klassen, Interfaces, Funktionen und Variablen in eigenen Namensräume
- Unterscheidung zwischen internen und externen Modulen
 - Interne Module: Modul-Spezifikation aus ECMAScript 6
 - Externe Module: JavaScript-Bibliothek z.B. mit CommonJS



TypeScript

- TypeScript kann alle JavaScript-Aspekte nutzen
- Ein Programm kann auch schrittweise von JavaScript nach TypeScript überführt werden
- Im Unternehmensumfeld wird in der Regel TypeScript anstelle von JavaScript verwendet
 - Bessere Typsicherheit → weniger Probleme
 - Compiler findet bereits fehler
 - Syntax-Highlighting mit Fehlerhinweisen
- Jedes .js-Datei könnte in .ts umbenannt werden → kein Problem
- TypeScript ist Case-Sensitiv
- Semikolon ist wie bei JavaScript nicht notwendig
- Kommentare sind wie bei Java

Setup

TypeScript: Setup

- TypeScript installieren

```
// Lokal  
npm install typescript --save-dev  
// Global  
npm install -g typescript
```

- TypeScript-Compiler ist dann im node_modules-Verzeichnis
- Je nach Installationsart (global oder lokal) ist Starten von Compiler anders

```
// Wenn lokal installiert  
npx tsc  
// Wenn global installiert  
tsc
```

- Anlegen einer Basiskonfiguration für TypeScript-Compiler

```
npx tsc --init
```

TypeScript-Konfiguration

- Basiseinstellungen für den Compiler

```
tsconfig.json > ...
1  {
2    "compilerOptions": {
3      "target": "es2015",
4      "lib": [
5        "dom",
6        "dom.iterable",
7        "esnext"
8      ],
9      "allowJs": true,
10     "skipLibCheck": true,
11     "esModuleInterop": true,
12     "allowSyntheticDefaultImports": true,
13     "strict": true,
14     "forceConsistentCasingInFileNames": true,
15     "noFallthroughCasesInSwitch": true,
16     "module": "esnext",
17     "moduleResolution": "node",
18     "resolveJsonModule": true,
19     "isolatedModules": true,
20     "noEmit": true,
21     "jsx": "react-jsx"
22   },
23   "include": [
24     "src"
25   ]
26 }
27
```

TypeScript: Übersetzen und ausführen, ts-node.js

- Für Ausführen von TypeScript-Modulen unter node sind 2 Schritte notwendig
 - TypeScript nach JavaScript transkompilieren
 - JavaScript ausführen
- Alternativ kann ts-node.js verwendet werden
 - <https://www.npmjs.com/package/ts-node>
 - ts-node macht beides mit einem Schritt (und mehr...)

TypeScript: Beispiel

- Einfaches TypeScript-Beispiel

Typdeklaration



```
var message:string = "Hello World"
console.log(message)
```

- Compiler erzeugt JavaScript-Code
- Erzeugte JavaScript-Code

```
var message = "Hello World";
console.log(message);
```

- TypeScript erzeugt keinen besseren JavaScript-Code
- Der Vorteil besteht aus der Typsicherheit, beim Erzeugen des JavaScript-Codes

TypeScript: Beispiel

Beispiel:

- Klasse mit zwei Attributen/ Methode, die Objekt verarbeitet
- Instanz wird angelegt und Methode angewandt

```
1  type User = {  
2      name: string;  
3      age: number;  
4  };  
5  
6  function isAdult(user: User): boolean {  
7      return user.age >= 18;  
8  }  
9  
10 const justine: User = {  
11     name: 'Justine',  
12     age: 23,  
13 };  
14  
15 const isJustineAnAdult: boolean = isAdult(justine);  
16  
17 console.log("Hallo")  
18
```

TypeScript: Beispiel

Kompilation:

```
tsc test.ts
```

Ergebnis:

```
1  function isAdult(user) {  
2      return user.age >= 18;  
3  }  
4  var justine = {  
5      name: 'Justine',  
6      age: 23  
7  };  
8  var isJustineAnAdult = isAdult(justine);  
9  console.log("Hallo");  
10
```

- Normaler JavaScript-Code
- Auch hier:
 - Keine bessere Typsicherheit
 - Nur in TypeScript-Modulen ist der Unterschied!

Grundlagen

TypeScript: Datentypen

- number: 64-Bit-Double-Wert, für Integer-Werte und Dezimalzahlen
- string: Unicode-Zeichen
- boolean: true oder false
- void: Für Funktionen, die keinen Rückgabewert haben
- null: Wenn kein Wert gesetzt ist
- undefined: nicht initialisierte Variable

- Null und undefined:
 - Es gibt einen Unterschied
 - Undefined ist eine Variable, die mit undefined initialisiert wurde
 - Null ist eine Variable, die auf ein Objekt zeigt, das „undefined“ ist

TypeScript: Scope

- Scope von Variablen wie in Java
 - Globale Variablen
 - Instanz-Variablen
 - Statische Variablen
 - Methoden-Variablen

```
1
2  var global_num = 12
3  class Numbers {
4      num_val = 13;
5      static sval = 10;
6
7      storeNum():void {
8          var local_num = 14;
9      }
10 }
11 console.log("Global num: "+global_num)
12 console.log(Numbers.sval)
13 var obj = new Numbers();
14 console.log("Global num: "+obj.num_val) |
```

TypeScript: Neue Konstrukte

- Methodensignatur
- Typinferenz
- Type Erasure
- Interfaces
- Aufzählungstyp
- Generische Programmierung
- Namensräume
- Tupel
- Async/Await

TypeScript: Methodensignatur

- Abweichend von Java
- Type und Variablennamen umgekehrt
- Return-Wert hinten
- Datentypen wie in JavaScript

```
function add(left: number, right: number): number {  
    return left + right;  
}
```

- Optionale Parameter

Optional



```
function disp_details(id:number,name:string,mail_id?:string) {  
    console.log("ID:", id);  
    console.log("Name",name);  
  
    if(mail_id!==undefined)  
        console.log("Email Id",mail_id);  
}
```

TypeScript: Klassen

- Sehr ähnlich wie in Java

```
class Person {  
    private name: string;  
    private age: number;  
    private salary: number;  
  
    constructor(name: string, age: number, salary: number) {  
        this.name = name;  
        this.age = age;  
        this.salary = salary;  
    }  
  
    toString(): string {  
        return `${this.name} (${this.age}) (${this.salary})`; // As of version 1.4  
    }  
}
```

TypeScript: Interfaces

- Definition ähnlich wie in Java
- Jedoch strukturelle Typing wird angewandt

```
firstSteps > TS interface.ts > ...  
1  
2 interface Component {  
3     id: string  
4 }  
5  
6 var myComponent: Component = {  
7     id: "Test"  
8 }  
9
```

- Es können Instanz-Variablen deklariert werden
- Es wird beim Typing geprüft, ob die Struktur stimmt

TypeScript: Generics

- Sehr ähnlich wie in Java

```
function doSomething<T>(arg: T): T {  
    return arg;  
}
```

TypeScript: Schleifen

- Sehr ähnlich wie in Java

```
var num:number = 0
var count:number = 0;

for(num=0;num<=20;num++) {
    if (num % 2==0) {
        continue
    }
    count++
}

console.log (" The count of odd values between 0 and 20 is: "+count)
```

TypeScript: Wenn Wert undefined sein kann

- Wenn ein Wert „undefined“ sein kann, muss das beim Typ entsprechend angegeben werden
- Bei der Übergabe an Funktionen kann es notwendig sein, den Optional-Parameter („?“) zu verwenden

```
firstSteps >  undefined.ts > ...  
1  
2 class Test {  
3   value: number | undefined  
4 }  
5  
6 var test: Test | undefined = new Test();  
7 test.value = 2;  
8  
9 if(test.value === 2)  
10 {  
11   test = undefined  
12 }  
13  
14 console.log("Value: " + test?.value)
```



TypeScript: Enum und Funktionsdeklaration

- Es können auch Enums und Funktionsdeklarationen verwendet werden
- Funktionsdeklarationen: wenn Funktionen weitergereicht werden sollen

```
export enum MessageType {  
    INFO,  
    WARN,  
    ERROR  
}  
  
export type ShowMessageToastFunction = {  
    (messageType: MessageType, title: string, text: string, autoHide: boolean): void  
}
```

REST-Server mit Node, Express und TypeScript

REST-Server mit TypeScript

- TypeScript hilfreich bei Umsetzung von Anwendungen wie REST-Server oder React-Anwendungen
- Typsicherheit und bessere Nutzung von Vererbung
- Notwendige Anpassungen
 - TypeScript-Module zu Modul hinzufügen
 - TypeScript-Konfiguration ergänzen
 - Anpassen von Start-Skript

TypeScript-Module hinzufügen

- TypeScript-Module als Dev-Dependencies installieren

```
npm i --save-dev @types/cors @types/express @types/debug source-map-support  
tslint typescript
```

```
"license": "ISC",  
"dependencies": {  
  "cors": "^2.8.5",  
  "debug": "^4.3.4",  
  "express": "^4.18.1",  
  "express-winston": "^4.2.0",  
  "winston": "^3.8.1"  
},  
"devDependencies": {  
  "@types/cors": "^2.8.12",  
  "@types/debug": "^4.1.7",  
  "@types/express": "^4.17.13",  
  "source-map-support": "^0.5.21",  
  "tslint": "^6.1.3",  
  "typescript": "^4.7.4"  
}  
}
```

TypeScript-Konfiguration hinzufügen

- Anlegen von tsconfig.json im Root-Verzeichnis
- Angaben für Compiler
 - Welcher Code soll erzeugt werden?
 - Welche Art von Modul ist es?
 - Wo soll der erzeugte Code abgelegt werden?

```
tsconfig.json > ...
1  {
2      "compilerOptions": {
3          "target": "es2016",
4          "module": "commonjs",
5          "outDir": "./dist",
6          "strict": true,
7          "esModuleInterop": true,
8          "inlineSourceMap": true
9      }
10 }
```

Start-Skript anpassen

- Start-Skript muss
 - Transkompilation durchführen
 - Im erzeugten Code das zentrale Modul starten (app.js)

```
"scripts": {  
  "start": "tsc && node --unhandled-rejections=strict ./dist/app.js",  
  "debug": "export DEBUG=* && npm run start",  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

Umsetzung der Routen

- Nutzen von Vererbung kann genutzt werden, um Programmieraufwand zu reduzieren
- Beispiel:
 - Basis-Klasse für Routen, Services, etc. umsetzen
 - Ausprägung für jede Route über Instanzen abbilden

```
export abstract class AbstractRoute {
  app: express.Application;
  name: string;

  constructor(app: express.Application, name: string) {
    this.app = app;
    this.name = name;
    this.configureRoutes();
  }
  getName() {
    return this.name;
  }
  abstract configureRoutes(): express.Application;
}
```

Umsetzung der User-Route

- Routen können Struktur und Methoden erben
- Basis-Implementierungen (z.B Standard-CRUD-Methoden) können auch in Basis-Klasse umgesetzt werden

```
import { AbstractRoute } from '../AbstractRoute';
import express from 'express';

export class UsersRoutes extends AbstractRoute {
  constructor(app: express.Application) {
    super(app, 'UsersRoutes');
  }

  configureRoutes() {
    this.app.route(`/users`)
      .get((req: express.Request, res: express.Response) => {
        res.status(200).send(`List of users`);
      })
      .post((req: express.Request, res: express.Response) => {
        res.status(200).send(`Post to users`);
      });
  }

  ~~~~

  return this.app;
}
}
```

Umsetzung der app.js

- Zentrales Modul ist kaum geändert

```
import { AbstractRoute } from './routes/AbstractRoute';
import { UsersRoutes } from './routes/users/UsersRoute';
import debug from 'debug';

const app: express.Application = express();
const server: http.Server = http.createServer(app);
const port = 8080;
const routes: Array<AbstractRoute> = [];
const debugLog: debug.IDebugger = debug('app')
```

```
// initialize the logger with the above configuration
app.use(expressWinston.logger(loggerOptions));

// here we are adding the UserRoutes to our array,
// after sending the Express.js application object to have the routes added to our app!
routes.push(new UsersRoutes(app, "publicUsers"));
```

- Es kann jedoch sehr viel generischer programmiert werden...

```
server.listen(port, () => {
  routes.forEach((route: AbstractRoute) => {
    debugLog(`Routes configured for ${route.getName()}`);
  });
  // our only exception to avoiding console.log(), because we
  // always want to know when the server is done starting up
  console.log(runningMessage);
});
```

Umsetzung der einzelnen Routen mit minimalen Code

- In der User-Route muss nur noch die Methode der abstrakte Klasse zum Anlegen aller Routen aufgerufen werden

```
export class UsersRoutes extends AbstractRoute {  
  constructor(app: express.Application, resource: string) {  
    super(app, resource, 'UsersRoutes');  
  }  
  
  configureRoutes() {  
    super.configureRoutes();  
  }  
}
```

- Die einzige Anpassung besteht in dem Ändern der Resource-ID („publicUsers“)

```
// after sending the Express.js application object to  
routes.push(new UsersRoutes(app, "publicUsers"));
```

React-Anwendung mit TypeScript

React-Anwendung mit TypeScript

- Erzeugen der React-Anwendung über

```
npx create-react-app my-app --template typescript
```

- Anschließend sollte Anwendung über „npm start“ gestartet werden
- Anwendung identisch zu normaler Java-Script-React-Anwendung
- Erzeugter Code sehr ähnlich, jedoch mit ein paar Unterschieden
 - tsconfig.json (Konfiguration für Transkompilation)
 - **React-Komponenten müssen die Endung „.tsx“ haben**
 - Ansonsten wird HTML-Code als Fehler dargestellt
 - Dateien, die kein JSX zurückgeben, sollten die Endung „.ts“ haben

React-Anwendung mit TypeScript

- Unterschied bei Funktionsdeklaration

```
function App() {  
  ...  
}
```

- wird zu ...

```
function App(): React.ReactNode {  
  ...  
}
```

React-Anwendung mit TypeScript

- Wenn Komponenten JSX enthalten, bzw. zurückgeben, muss vorher transkompiliert werden
- Dazu muss Modul Endung „.tsx“ haben
- Hinweis für Transkompiler
- Ansonsten Fehlermeldungen beim JSX-Code